ECEN 5793
Digital Image Processing

Nate Lannan

Agenda

Image acquisition

Ideal interpolation

Linear interpolation

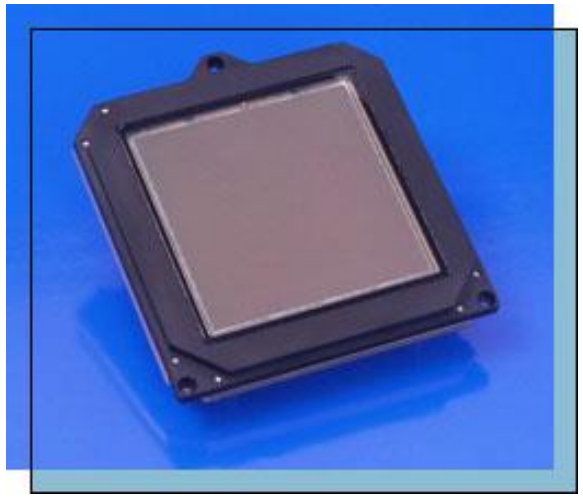
Bicubic interpolation

Super-resolution

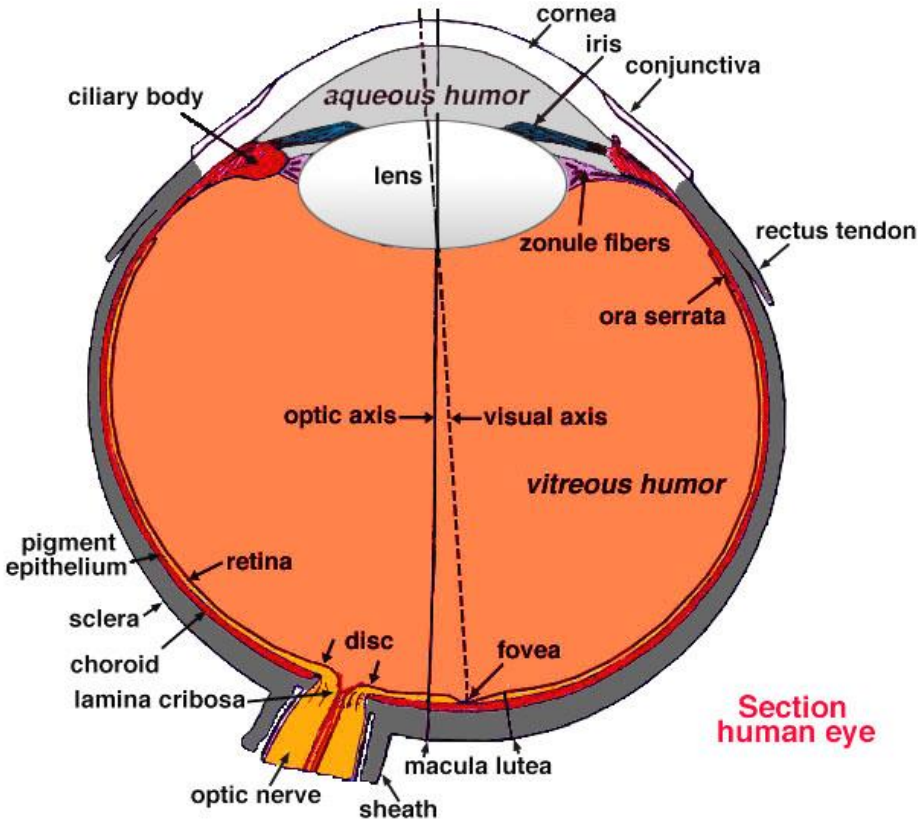
Questions

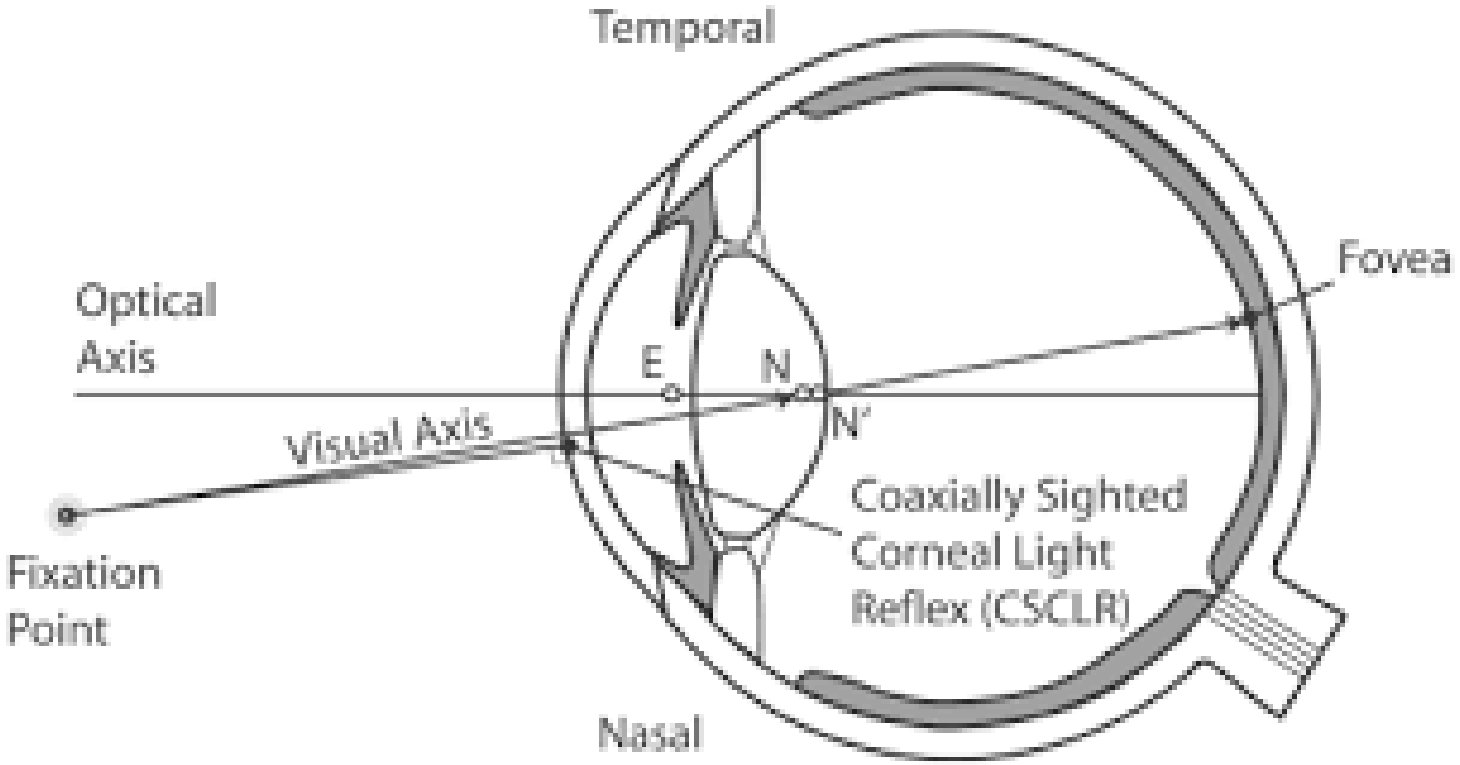
Is the human visual system (HVS) telling the truth all the time?

What are the two main issues about digital image acquisition?



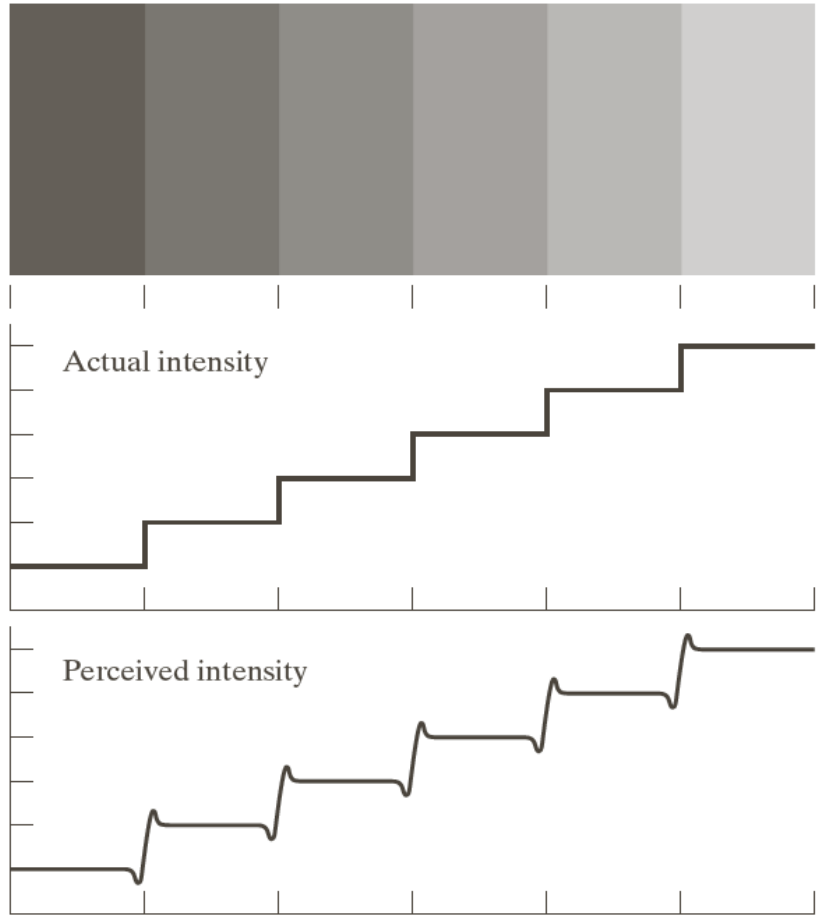
Review of Eye Structure





Mach Bands

The HVS tends to undershoot or overshoot around the boundary of regions of different intensities.



a
b
c

FIGURE 2.7 Illustration of the Mach band effect. Perceived intensity is not a simple function of actual intensity.

Visual Illusion

a b
c d

FIGURE 2.9 Some well-known optical illusions.

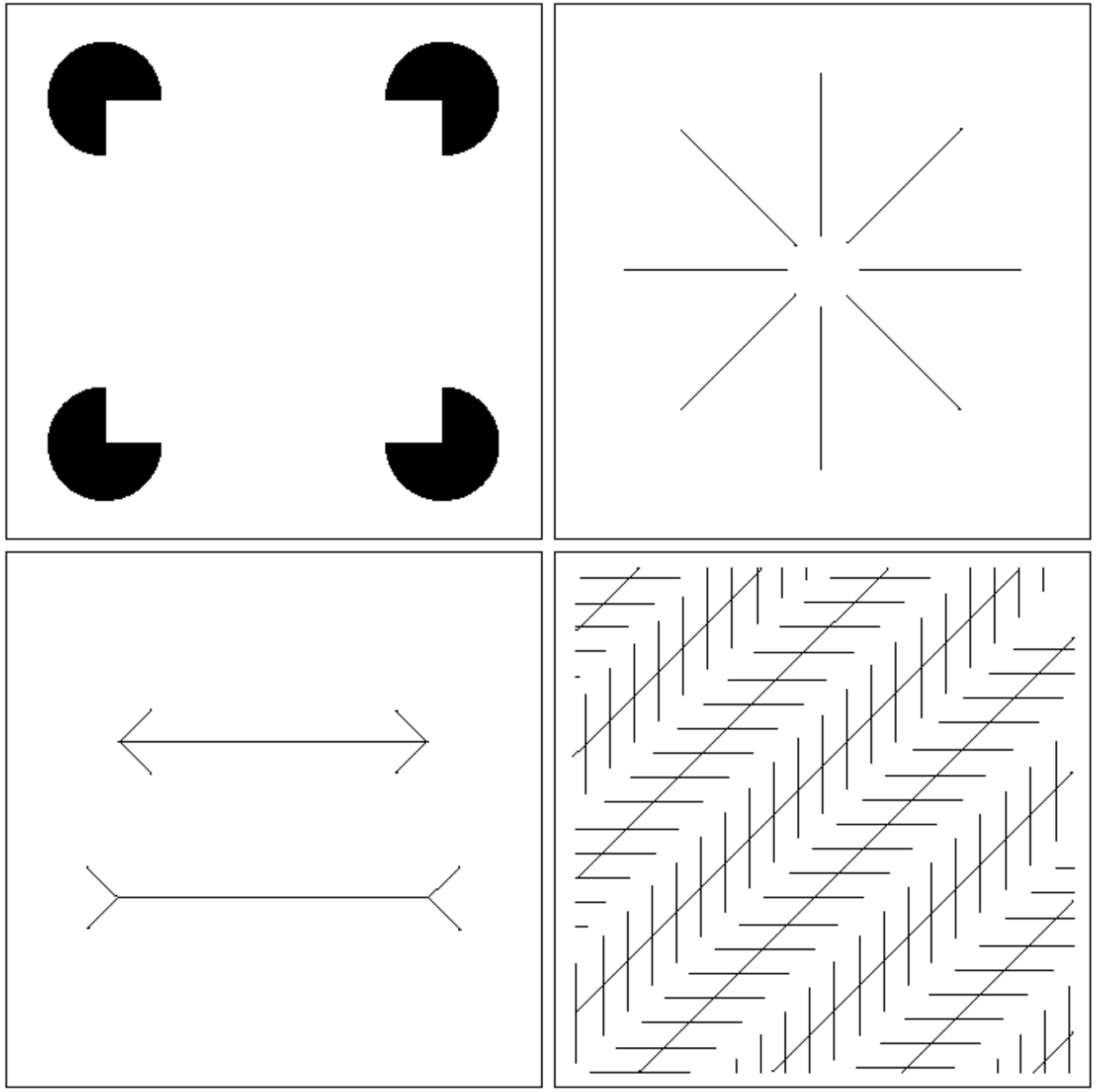
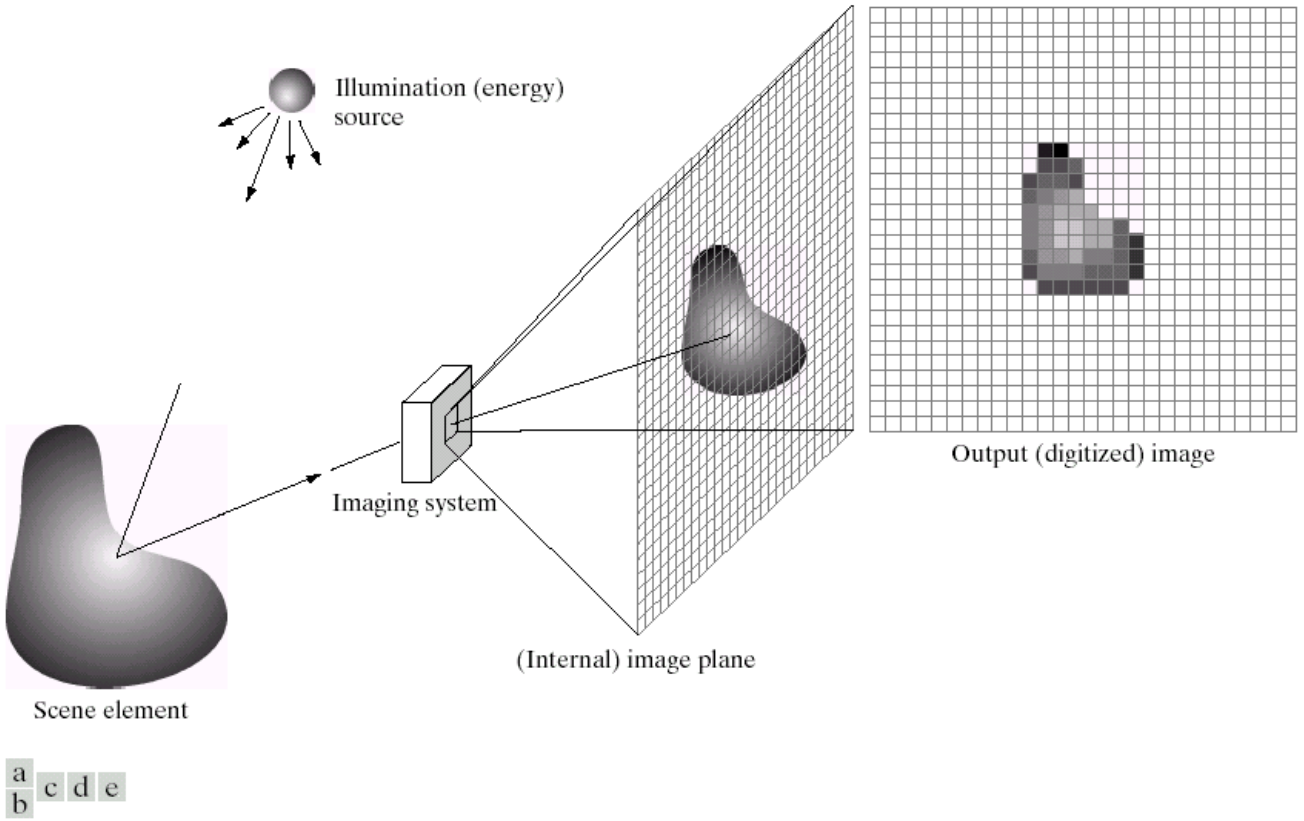


Image Acquisition: Sampling and Quantization



a b c d e

FIGURE 2.15 An example of the digital image acquisition process. (a) Energy ("illumination") source. (b) An element of a scene. (c) Imaging system. (d) Projection of the scene onto the image plane. (e) Digitized image.

Spatial Resolution

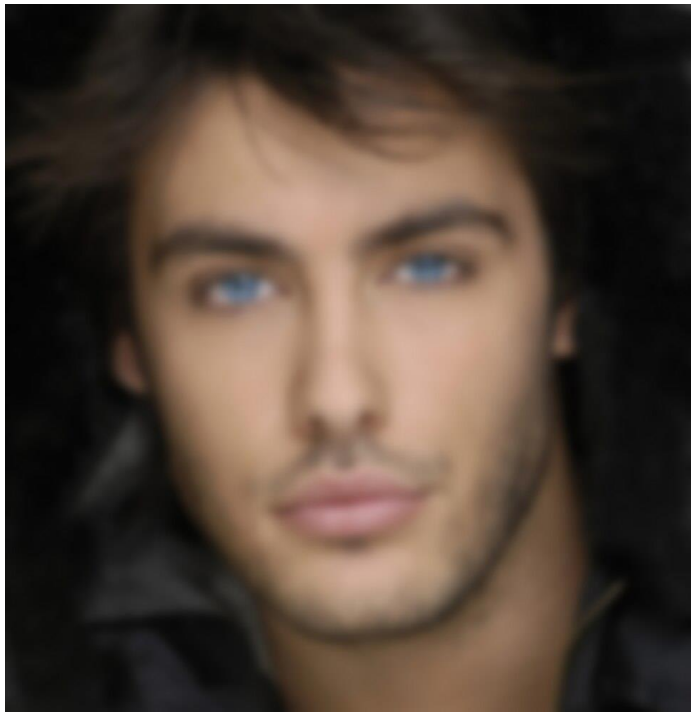
Just because an image has high resolution (i.e. pixel count) does not mean it is a quality image.

We need to relate the pixels to spatial units. The image itself does not tell the whole story.

Typical metrics

Printing – dpi, ppi

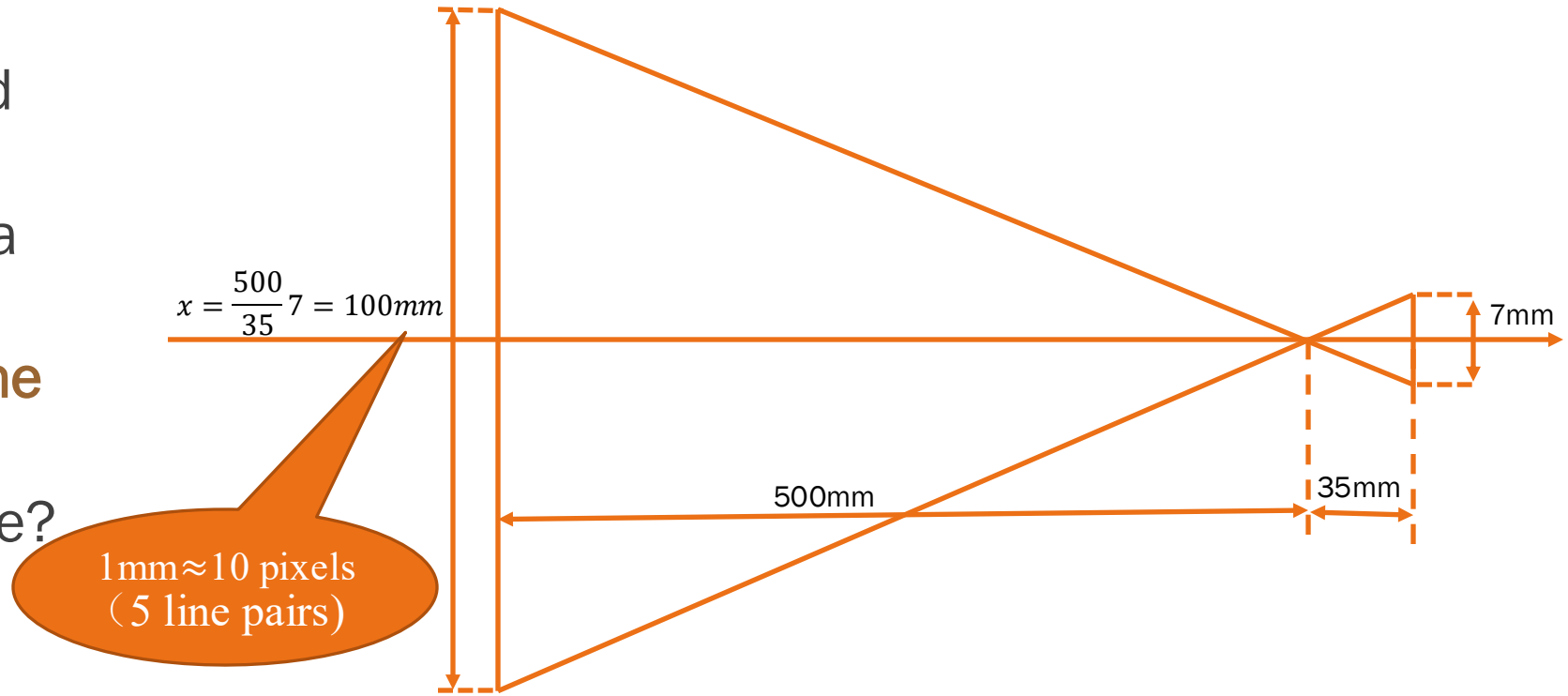
Imaging – LP/mm



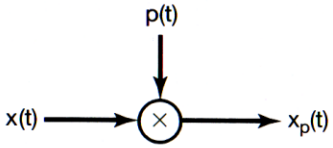
These images have the same pixel count but different spatial resolution

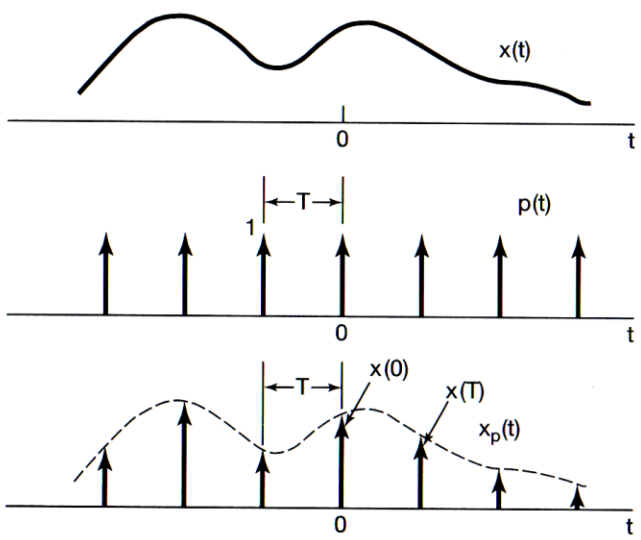
Example of Sampling

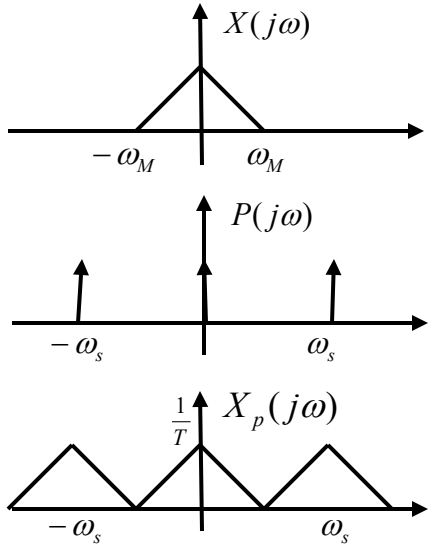
A CCD (Charge-coupled device) camera chip of dimensions 7x7 mm, and having 1024x1024 elements, is focused on a square, flat area, located 0.5m away. How many **line pairs per mm** will this camera be able to resolve? The camera is equipped with a 35-mm lens.



Review of Nyquist Theorem







$x(t)$ is a bandlimited signal

$|X(j\omega)| = 0, |\omega| < \omega_M$

$$P(j\omega) = \frac{2\pi}{T}\sum_{k=-\infty}^{\infty}\delta(\omega - k\omega_s)$$

$$(\omega_s = \frac{2\pi}{T}, \text{sampling rate})$$

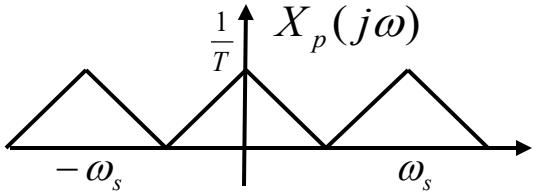
$$x_p(t) = x(t)p(t)$$

$$X_p(j\omega) = \frac{1}{2\pi}X(j\omega) * P(j\omega)$$

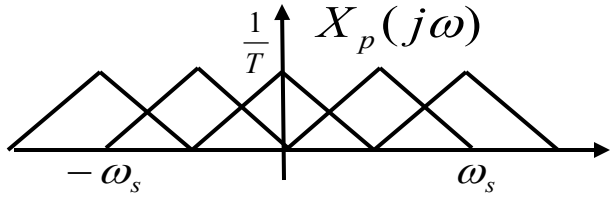
$\omega_s > 2\omega_M \rightarrow$ aliasing free

Two Sampling Conditions

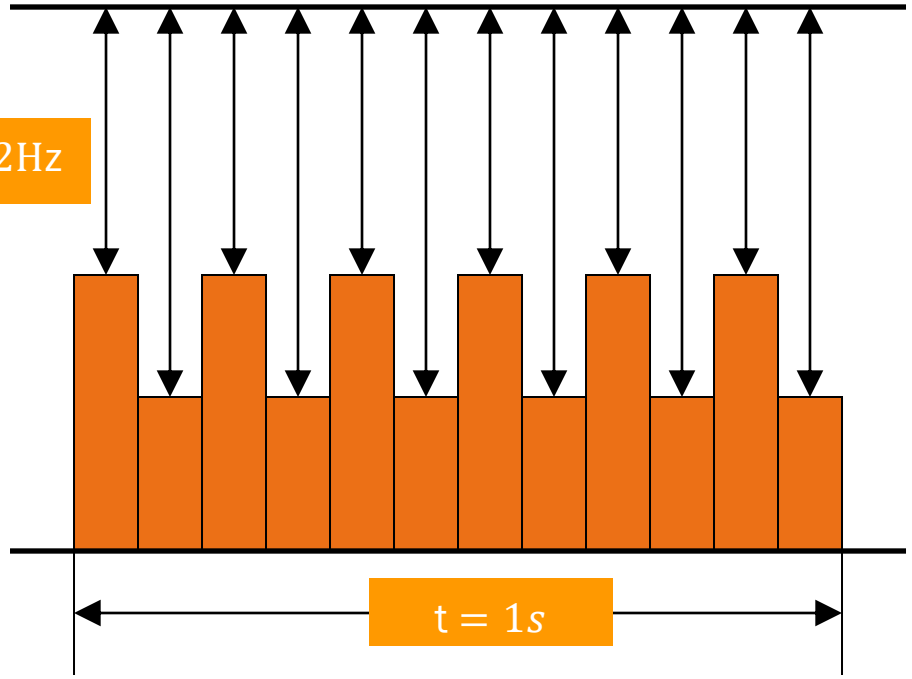
$\omega_s = 2\omega_M$



$\omega_s = \omega_M$

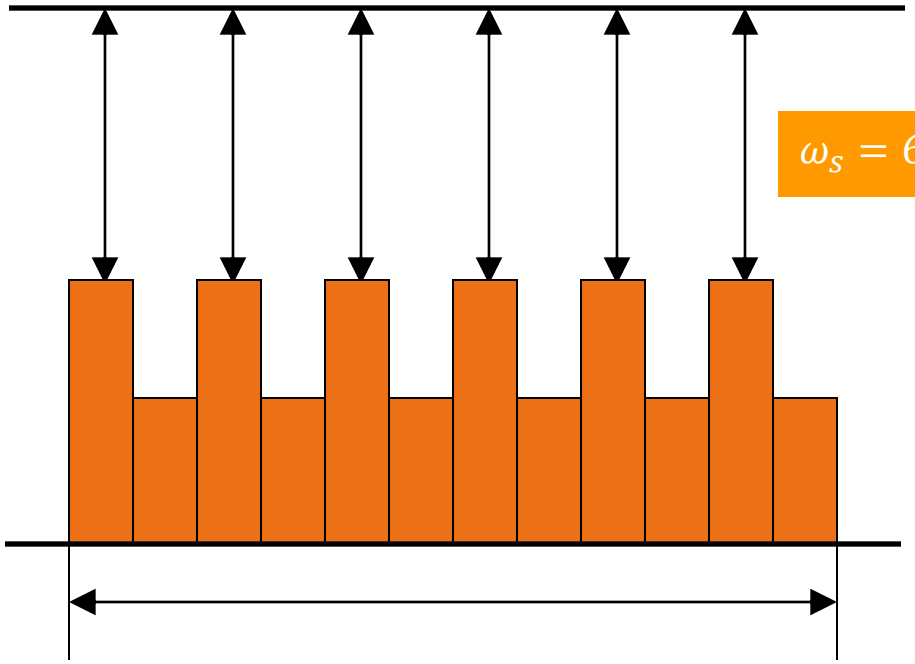


$\omega_s = 12\text{Hz}$



$\omega_M = 6\text{Hz}$

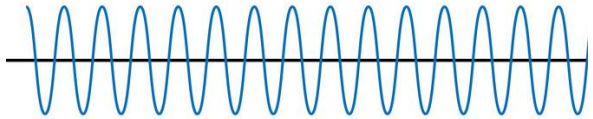
$\omega_s = 6\text{Hz}$



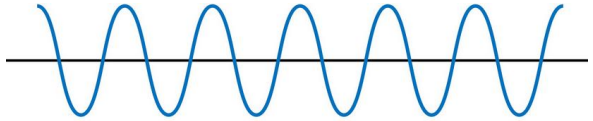
Frequency in Images

Spatial frequency has content like time-based signals

High Frequency



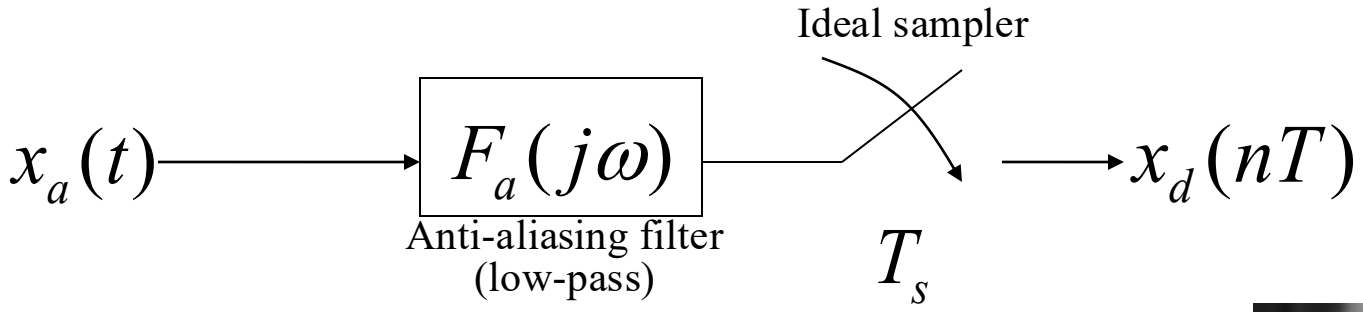
Low Frequency



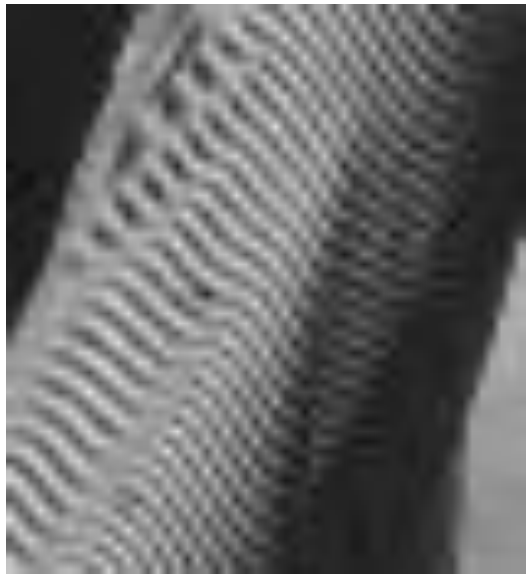
What is the high frequency content and low frequency content in this image?

Aliasing Problems

Anti-aliasing pre-filtering before sampling

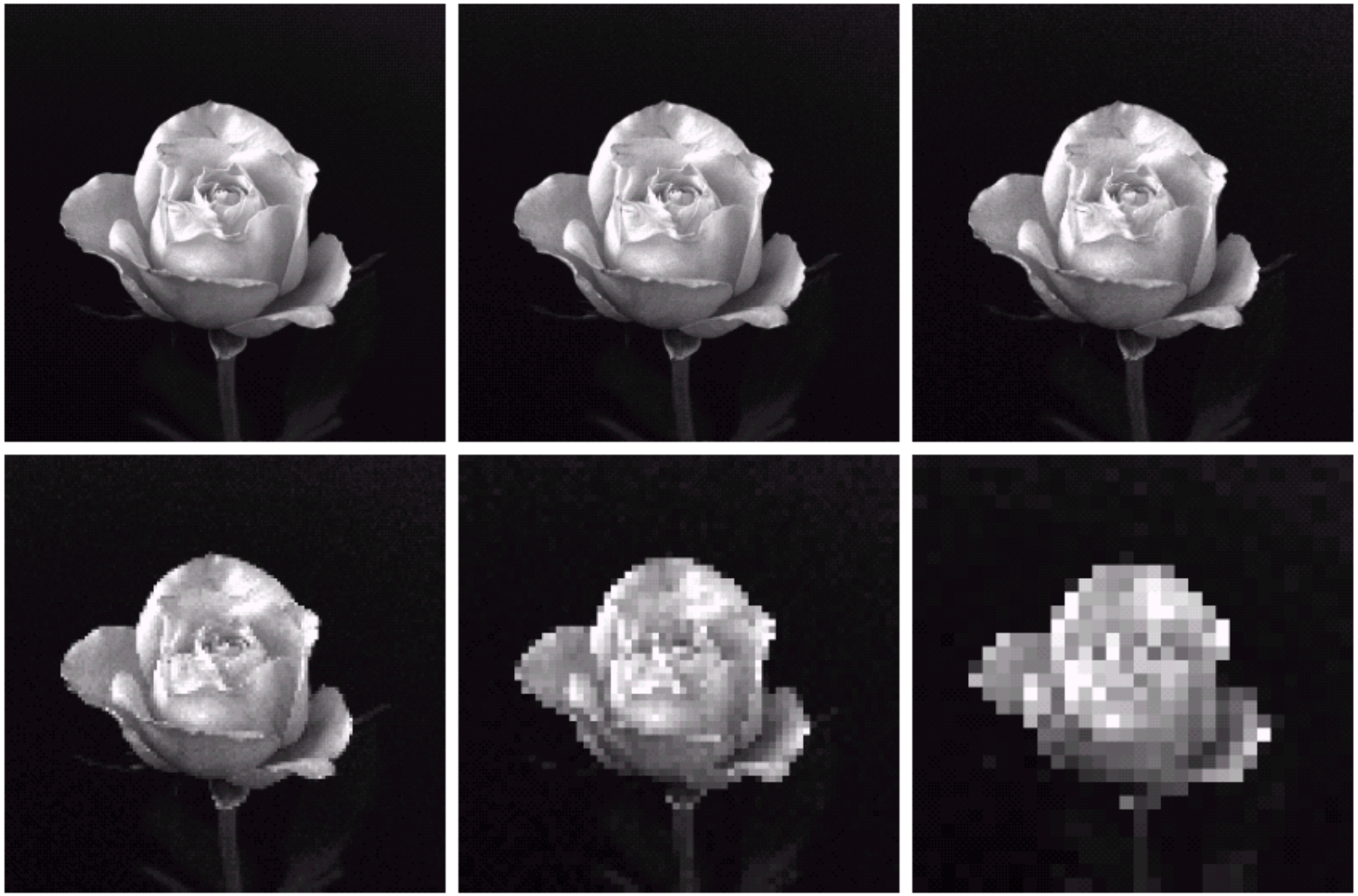


Original image



Sampling with aliasing

Sampling with anti-aliasing



a b c
d e f

FIGURE 2.20 (a) 1024 × 1024, 8-bit image. (b) 512 × 512 image resampled into 1024 × 1024 pixels by row and column duplication. (c) through (f) 256 × 256, 128 × 128, 64 × 64, and 32 × 32 images resampled into 1024 × 1024 pixels.

a b
c d

FIGURE 2.20 Typical effects of reducing spatial resolution. Images shown at: (a) 1250 dpi, (b) 300 dpi, (c) 150 dpi, and (d) 72 dpi. The thin black borders were added for clarity. They are not part of the data.

Quantization

The sampling process requires decision about size *M* x *N*, and the quantization process decides the number of gray scales *L*.

Due to the consideration of processing, storage, and sampling hardware, *L* is typically an integer power of 2:

$$L = 2^k, \text{ and } f(x, y) \in [0, L-1]$$

The number of bits, b, required to store a digitized image is

$$b = M \times N \times k. \qquad b = N^2 k \quad (\text{when } M = N)$$

Number of Gray Levels

The number of gray levels mainly depends on both the contrast ratio (CR) of the environment and the contrast sensitivity about HVS.

	Contrast sensitivity		
	2%	1%	0.5%
Contrast ratio			
10 (office environment)	116 (~7 bits)	231 (~8 bits)	462 (~9 bits)
30 (living room, television)	172	342	682
100 (cinema theater)	232 (~8 bits)	463 (~9 bits)	923 (~10 bits)

$$f_N = 1 = \frac{(1+\Delta c)^N}{CR} \rightarrow CR = (1+\Delta c)^N$$

$$f_{k+1} = f_k(1+\Delta c)$$

$$f_2 = \frac{(1+\Delta c)^2}{CR}$$

$$f_1 = \frac{1+\Delta c}{CR}$$

$$f_0 = \frac{1}{CR}$$

Number of gray levels:

$$N = \frac{\log(CR)}{\log(1+\Delta c)}$$

$$\frac{\log(10)}{\log(1.02)} \approx 116.3$$


256 grayscales (8bpp)


128 grayscales (7bpp)

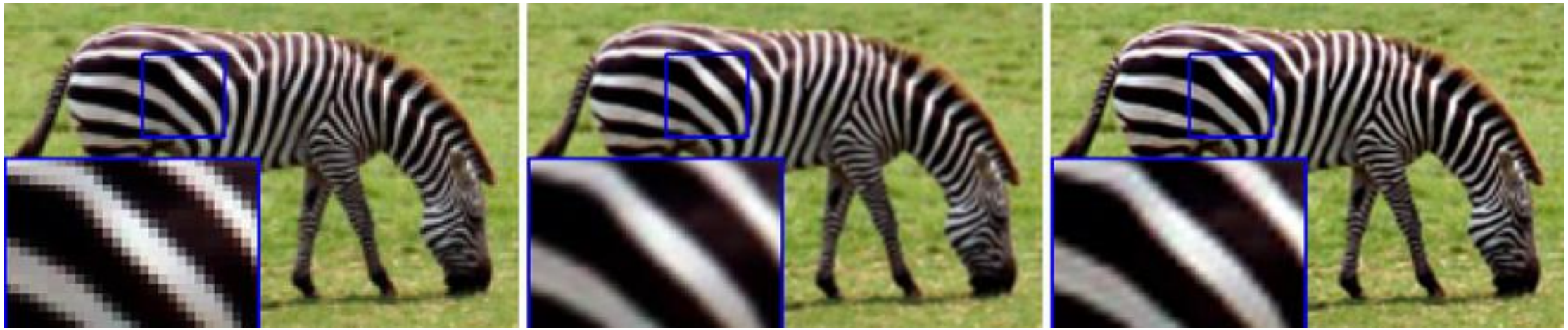

64 grayscales (6bpp)


32 grayscales (5bpp)


16 grayscales (4bpp)


8 grayscales (3bpp)

Interpolation



Questions

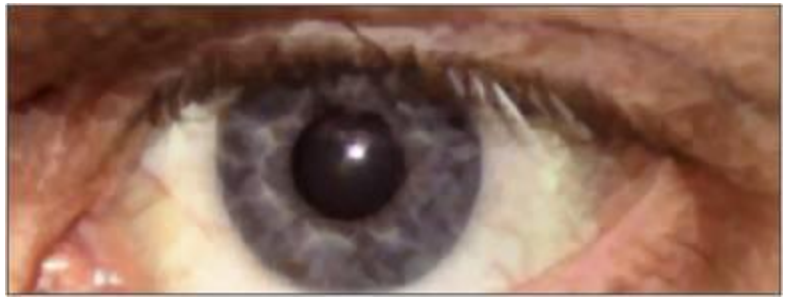
How to resize an image with best quality?

Original low-res image

Nearest neighbor

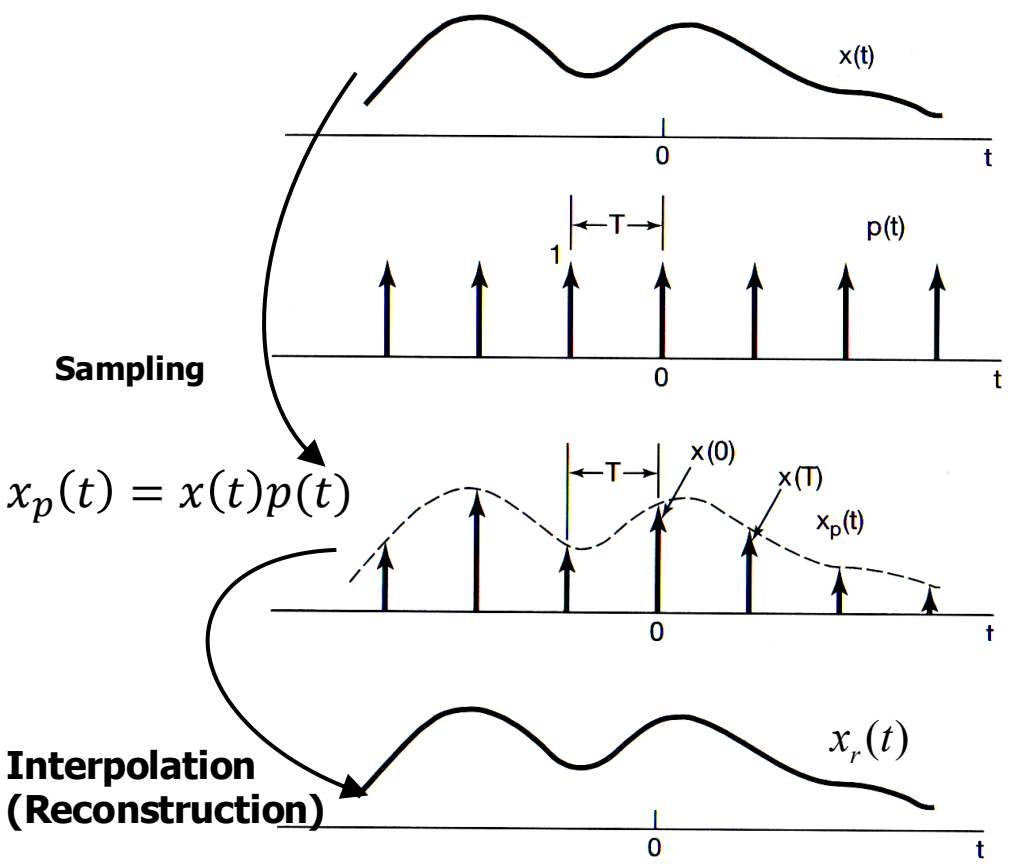
Bilinear interpolation

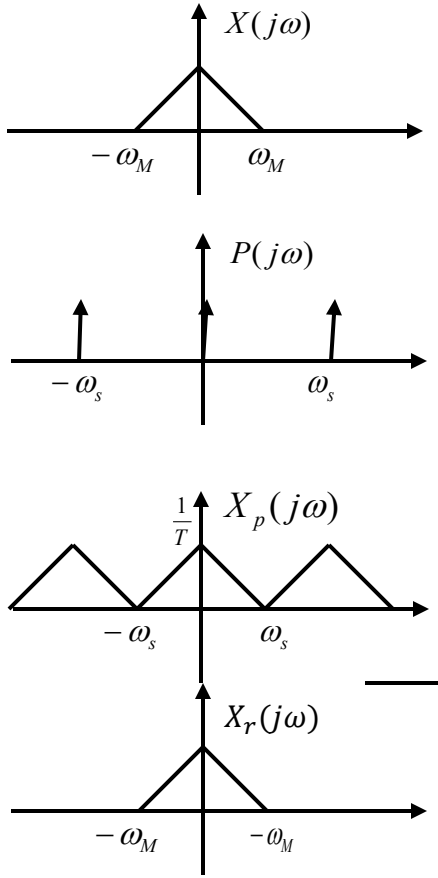
Bi-cubic interpolation

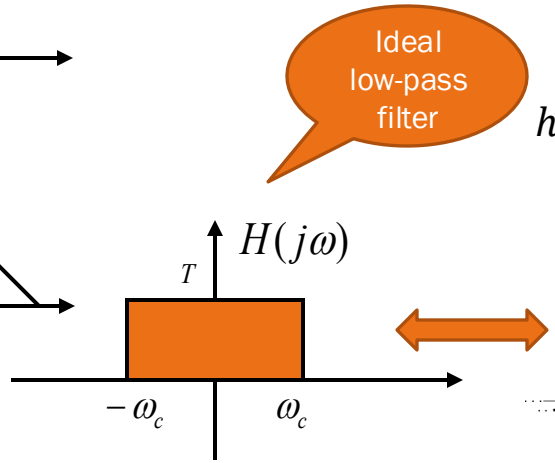


Model-based interpolation

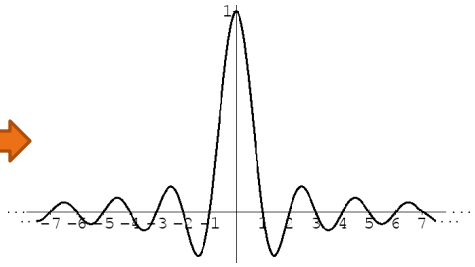
What is the ideal interpolation?







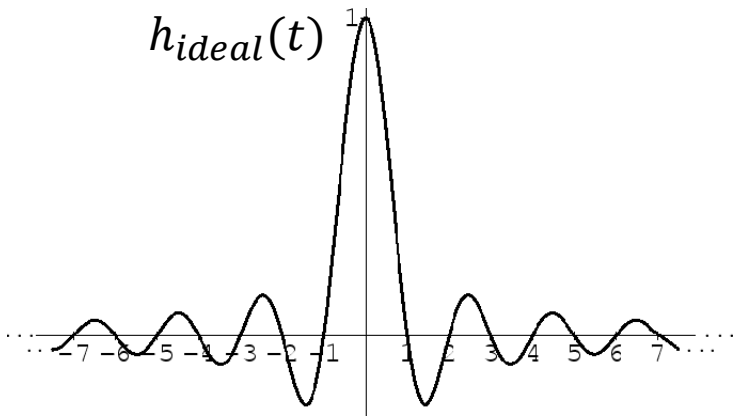
$$h_{ideal}(t) = \frac{T\sin(\omega_c t)}{\pi t}$$

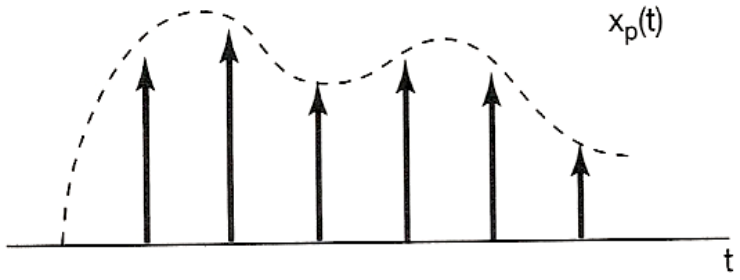


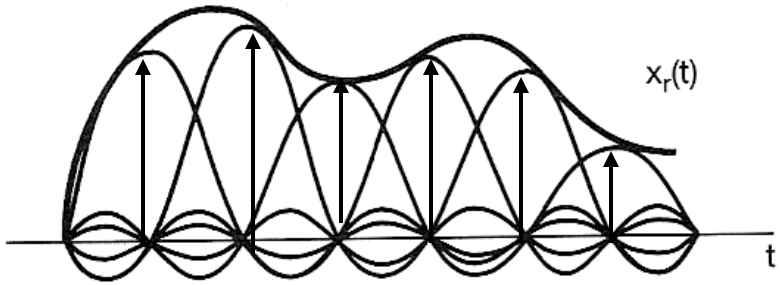
$$X_r(j\omega) = X_p(j\omega)H(j\omega)$$

Ideal Interpolation

$$X_r(j\omega) = X_p(j\omega)H(j\omega) \leftrightarrow x_r(t) = x_p(t) * h_{ideal}(t)$$



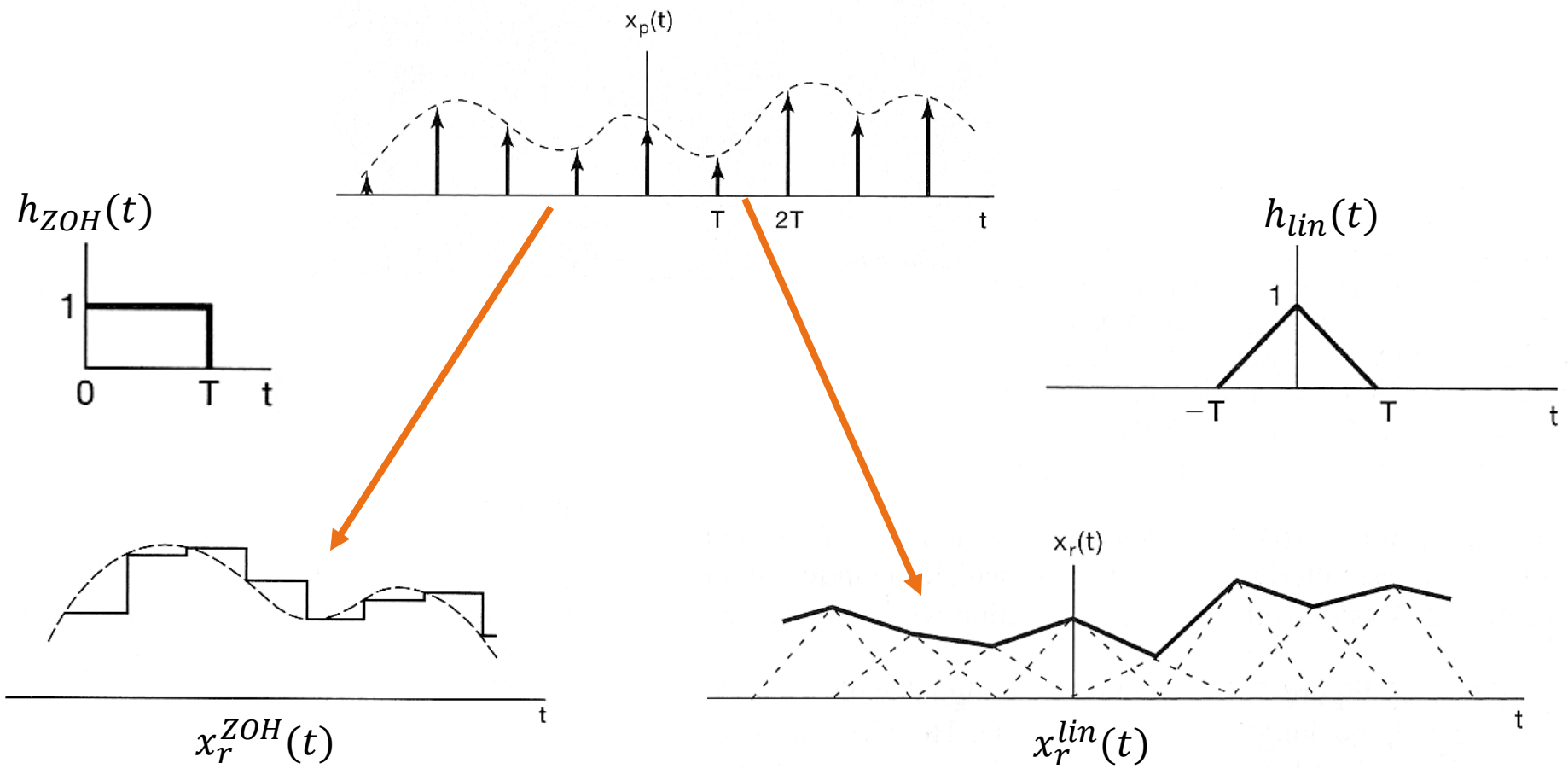




What is the implication here?

Dependency on infinity and non-causality make this not physically realizable

Zero-order Hold (ZOH) & Linear Interpolation



Zooming and Shrinking

Both zooming and shrinking are applied to digital images with two steps

- The creation of new pixel locations
- The assignment of gray levels to those new locations

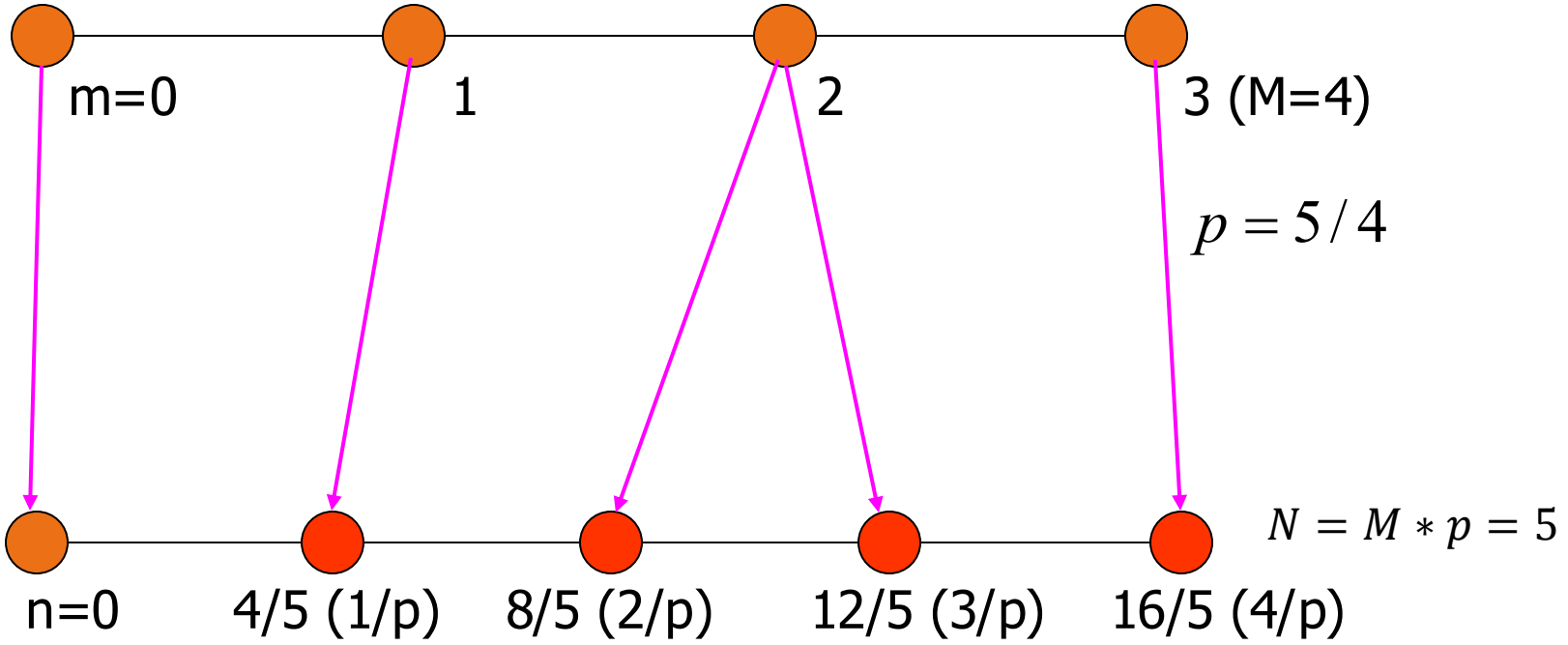
Pixel Interpolation is the process of using known data to estimate values at unknown locations.

- Nearest neighbor interpolation (ZOH)
- Pixel replication (special case of nearest neighbor interpolation)
- Bilinear interpolation
- Bicubic interpolation

Zooming and shrinking can be done in a similar manner.

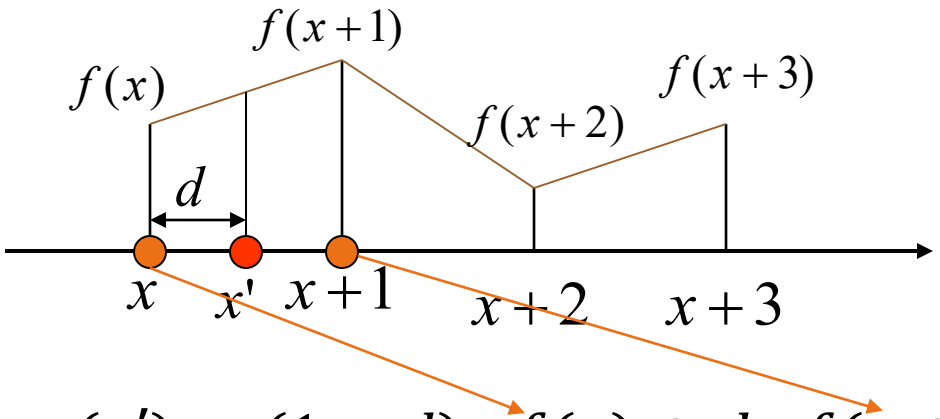
How to create new pixel locations?

In the zooming case, when ratio *p* is not integer, usually new pixels (red points) are created not on the grid.



Linear Interpolation (1-D)

The 1-D linear interpolator involves a linear model to compute the values of new samples as



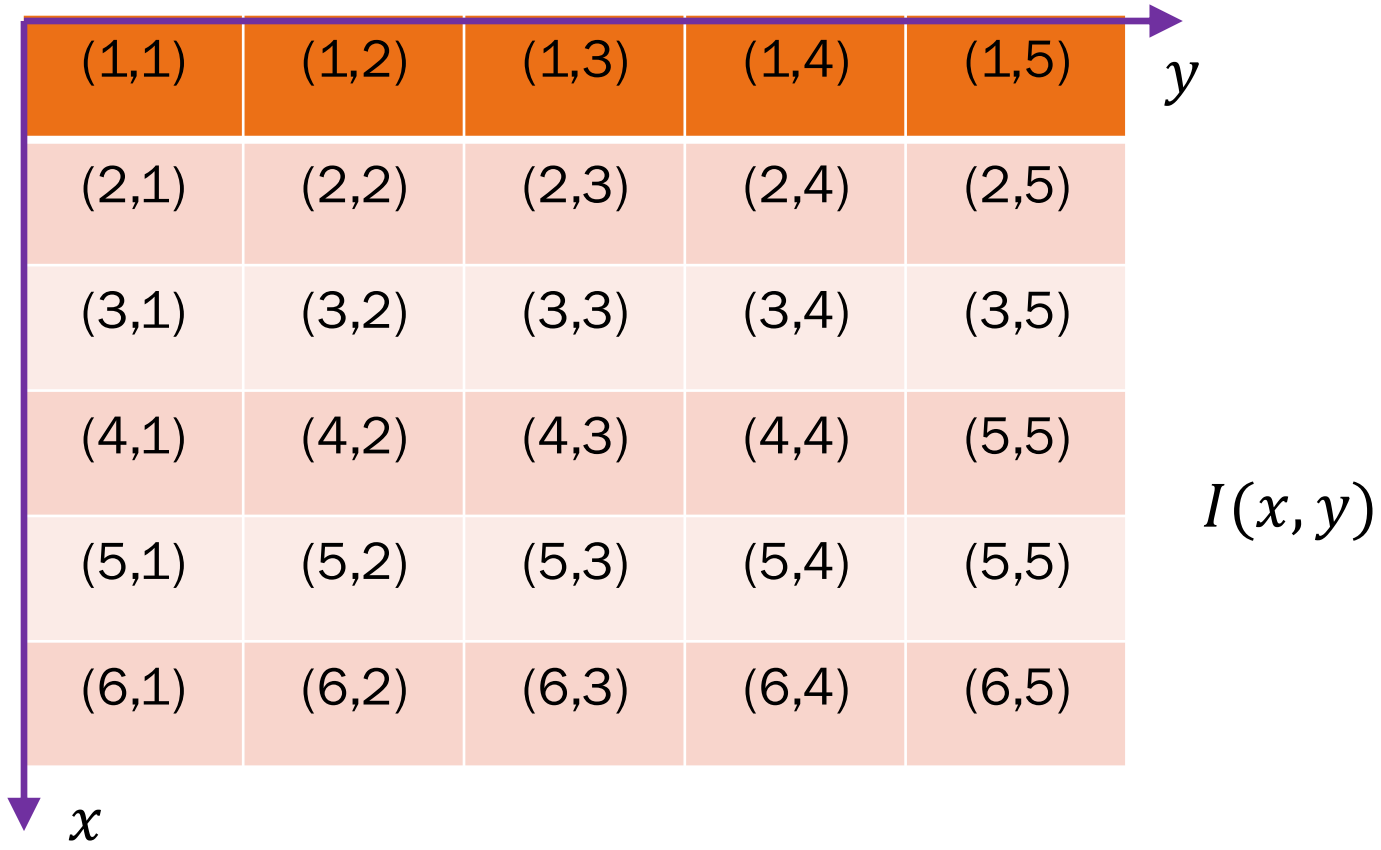
$$d = x' - x$$

$$x = \text{floor}(x')$$
$$x + 1 = \text{ceil}(x')$$

$$v(x') = (1 - d) \cdot f(x) + d \cdot f(x + 1)$$

$$v(x') = (1 - d) f(\text{floor}(x')) + d \cdot f(\text{ceil}(x'))$$

Coordinate System in Matlab



$I(x, y)$

Coordinate System in numpy

Coordinate swap

$I(y, x)$

A crash course in MATLAB

Image I/O
- imread() – input image
- imwrite() – output image
- imshow() – display image

Data types for intensity
- im2double() – convert values to double
- im2uint8(), im2uint16() – convert to an unsigned int
- mat2gray() – converts matrix to a gray scale image [0,1]

Indexing and geometry
- size() – return size of matrix
- ndims() – return number of dimensions of matrix
- meshgrid(), ndgrid() – replication of an array over matrix
- find() – find subscript where condition is satisfied in array
- sub2ind() – convert subscript notation to index

https://www.mathworks.com/help/images/

A crash course in a python stack

- Python3 – the more recent the better
- NumPy – library for matrix manipulation
- OpenCV – image I/O and classical computer vision
- Matplotlib – graphing and output image utilities
- scikit-image – image utilities

Simplest install:

- Install python - https://www.python.org/downloads/
- Install pip - https://pip.pypa.io/en/stable/installation/
- Use pip to install libraries (terminal)–
  - pip install numpy
  - pip install opencv-python
  - pip install matplotlib
  - pip install scikit-image

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
import skimage
```

https://docs.opencv.org/
https://numpy.org/doc/stable/
https://matplotlib.org/stable/api/index
https://scikit-image.org/docs/stable/

A crash course in a python stack

Image I/O

- cv2.imread() – loads BGR not RGB
  plt.imread()
  skimage.io.imread()
- cv2.imwrite()
  plt.imsave()
  skimage.io.imsave()
- plt.imshow()

Data types for intensity

- img.astype(np.float64)/max_value – explicit normalization for a double
- (img * 255).astype(np.uint8) – assumes [0,1] for converting to uint
- (img – img.min())/(img.max() – img.min()) – convert any matrix to gray scale

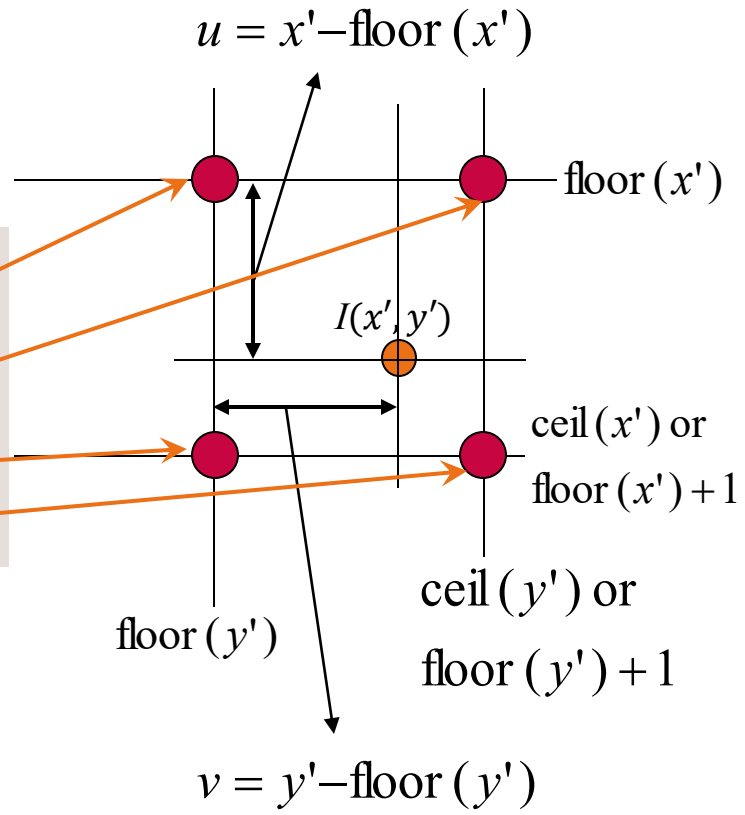
Indexing and geometry

- A.shape – return size of matrix
- A.ndim – return number of dimensions of matrix
- np.meshgrid(x,y) – replication of an array over matrix
- np.where() – find subscipt where condition is satisfied in array
- np.ravel_multi_index((r,c), sz) – convert subscript notation to index

Linear Interpolation (2-D)

Bilinear interpolation takes a weighted average of four pixels in the original image to the new pixel.

$$
\begin{aligned}
I(x', y') &= (1-u)(1-v) f(\text{floor}(x'), \text{floor}(y')) \\
&\quad + (1-u) \cdot v \cdot f(\text{floor}(x'), \text{ceil}(y')) \\
&\quad + u \cdot (1-v) \cdot f(\text{ceil}(x'), \text{floor}(y')) \\
&\quad + u \cdot v \cdot f(\text{ceil}(x'), \text{ceil}(y'))
\end{aligned}
$$

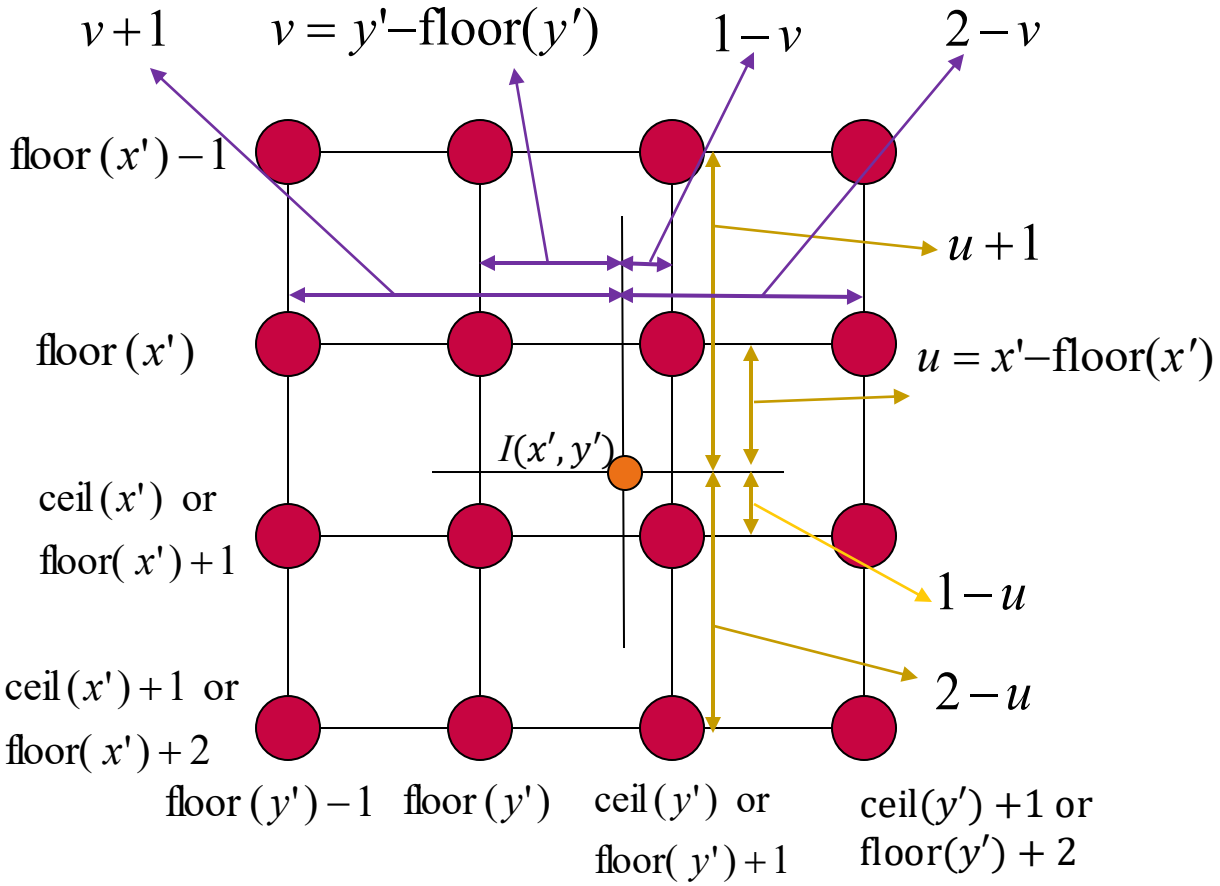


In programming, you have to use either floor(.) or ceil(.). But don't use them together. Why?

Bicubic Interpolation

Bicubic interpolation takes a weighted average of 16 pixels in the original image to the new pixel.

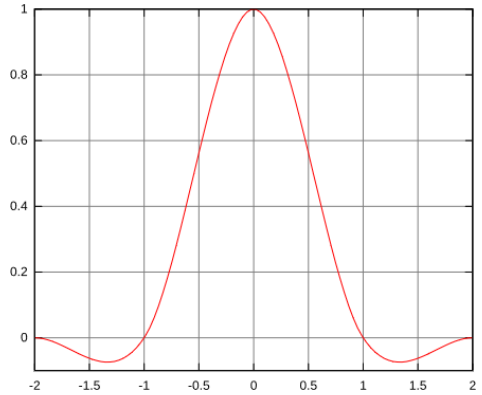
The contribution of each of the 16 neighboring pixels to the new interpolated point is defined by a bicubic function which is related the distance between the new point and each of 16 pixels (the closer distance, the stronger contribution)

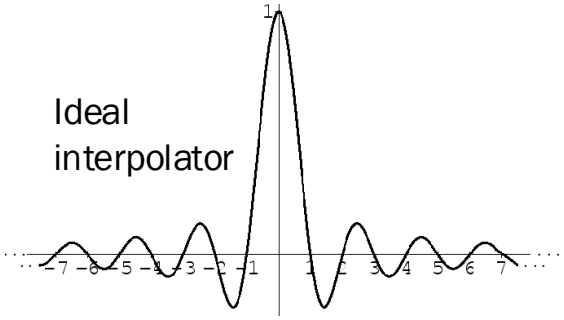


Bicubic Interpolation (Cont'd)

The interpolation kernel using the Ricker wavelet is given by

$$h(x) = \begin{cases} 1 - 2|x|^2 + |x|^3 & \text{for } 0 \le |x| \le 1 \\ 4 - 8|x| + 5|x|^2 - |x|^3 & \text{for } 1 < |x| \le 2 \\ 0 & \text{otherwise} \end{cases}$$





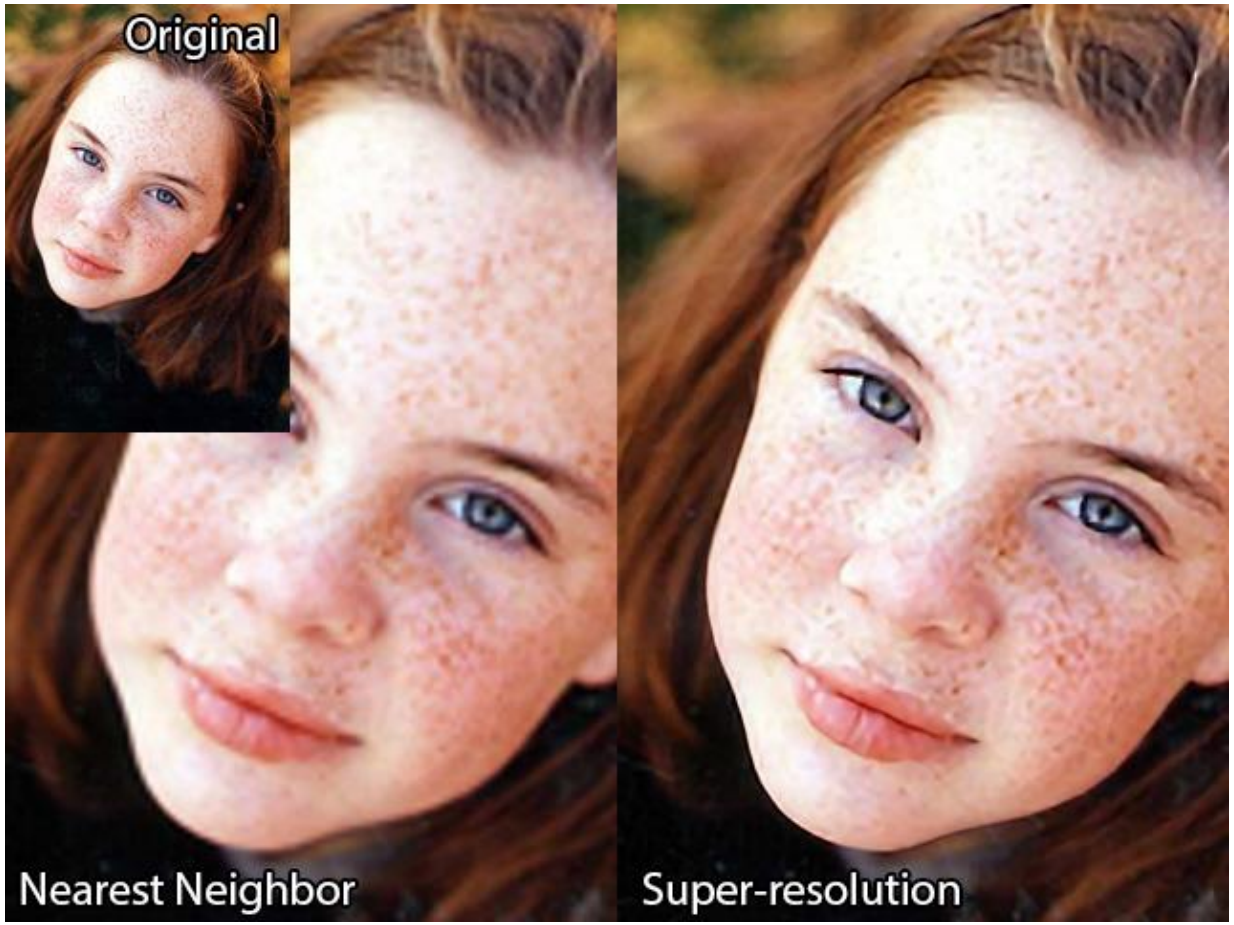
Then the 2D bicubic interpolation is computed as

$$I(x', y') = \sum_{m=1}^{4} \sum_{n=1}^{4} h(x_m - x') h(y_n - y') f(x_m, y_n)$$

Super-resolution vs. Interpolation (1)

Interpolation involves up-sampling the low-resolution image which may not recover sufficient high-frequency components, leading to blurred images.

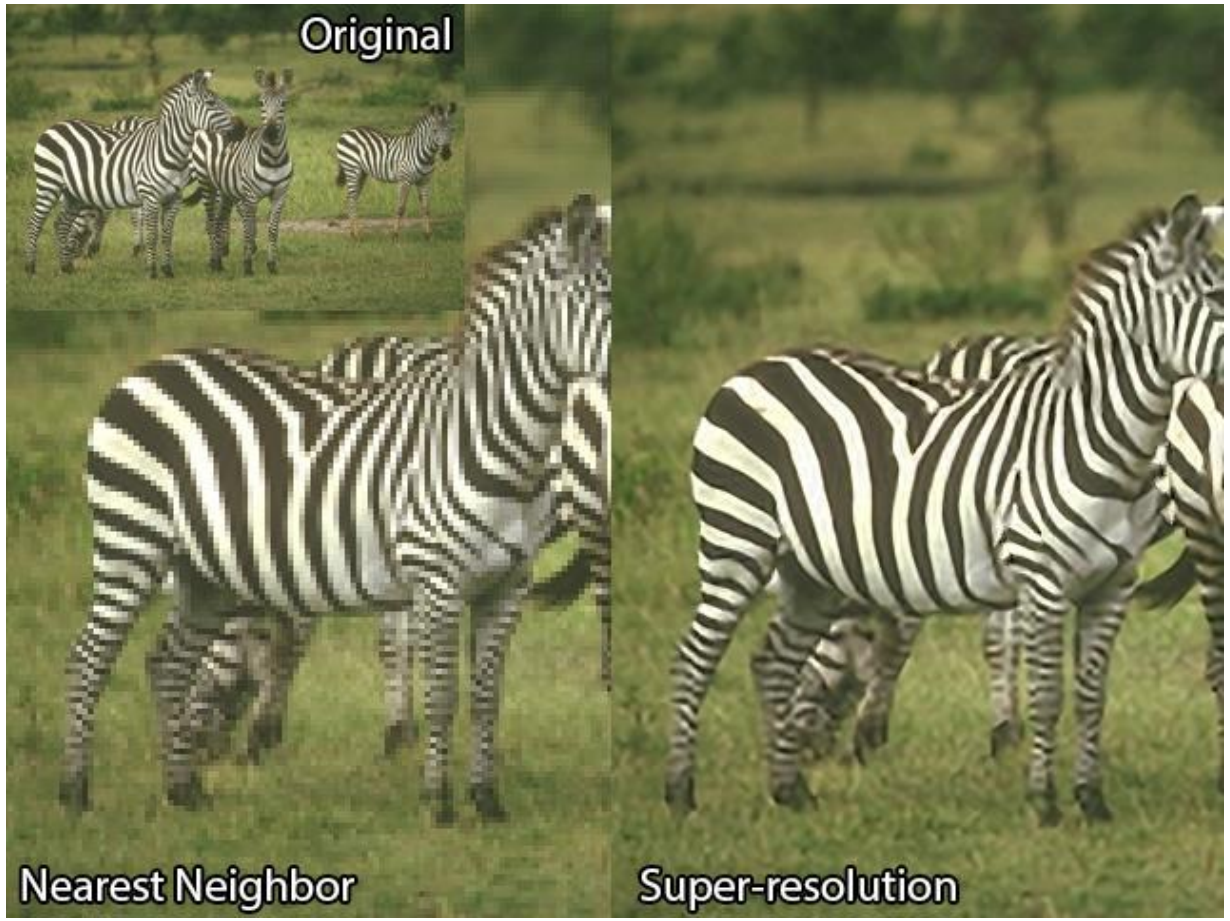
Super-resolution (SR) involves three major processes: interpolation, deblurring and denoising, leading to more detailed and sharper images.



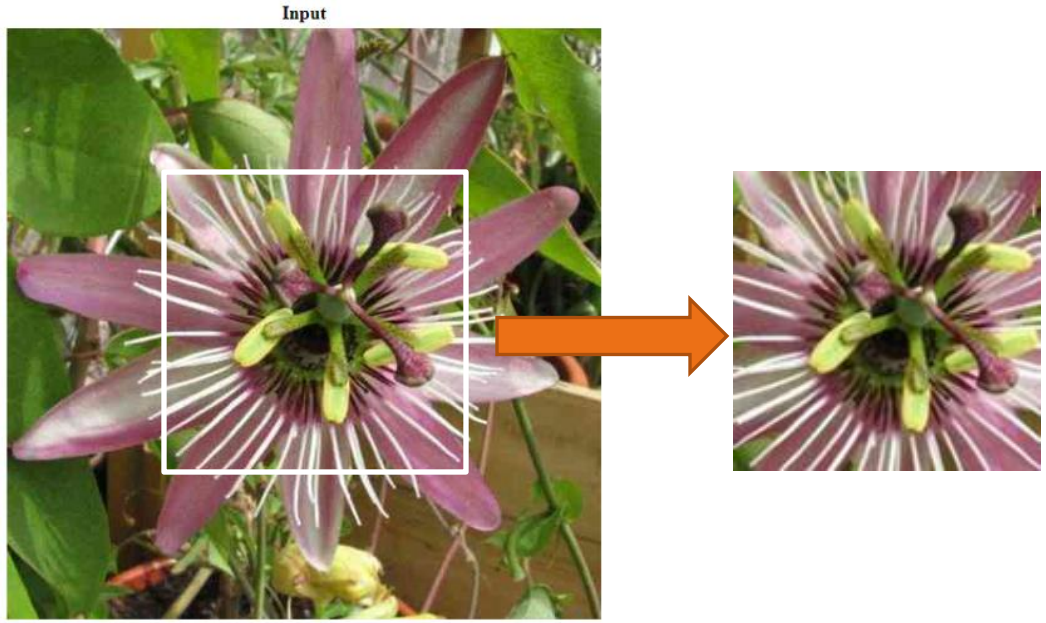
Super-resolution vs. Interpolation (2)

Interpolation involves up-sampling the low-resolution image which may not recover sufficient high-frequency components, leading to blurred images.

Super-resolution (SR) involves three major processes: interpolation, deblurring and denoising, leading to more detailed and sharper images.



Deep Learning for SR



Super Resolution using Keras:
https://github.com/MaokeAI/SRCNN-keras

https://medium.com/data-science/deep-learning-based-super-resolution-without-using-a-gan-11c9bb5b6cd5